Trivy Cheatsheet

- Copy to Clipboard
- Download PDF

Overview

Trivy is a comprehensive security scanner for containers, Infrastructure as Code (IaC), and file systems. It detects vulnerabilities, misconfigurations, secrets, and software licenses across multiple formats including Docker images, Kubernetes manifests, Terraform files, and more.

Key Features

- **Multi-format Support**: Docker images, Kubernetes, Terraform, CloudFormation, Helm charts
- **Vulnerability Detection**: OS packages, language-specific packages, and application dependencies
- **Misconfiguration Detection**: Security misconfigurations in IaC files
- **Secret Detection**: API keys, passwords, and other sensitive information
- **License Detection**: Software license compliance scanning
- **SBOM Generation**: Software Bill of Materials creation
- **CI/CD Integration**: GitHub Actions, GitLab CI, Jenkins support

Installation

Binary Installation

```
# Linux (x86_64)
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin

# macOS (Homebrew)
brew install trivy

# Windows (Chocolatey)
choco install trivy

# Windows (Scoop)
scoop install trivy
```

Package Manager Installation

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | sudo tee -a
/etc/apt/sources.list.d/trivy.list
sudo apt-get update
sudo apt-get install trivy

# RHEL/CentOS/Fedora
sudo vim /etc/yum.repos.d/trivy.repo
# Add repository configuration
sudo yum -y update
sudo yum -y install trivy

# Arch Linux
yay -S trivy-bin
```

Docker Installation

```
# Pull latest image
docker pull aquasec/trivy:latest

# Run with Docker
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v $HOME/Library/Caches:/root/.cache/ aquasec/trivy:latest image python:3.4-alpine
```

From Source

```
# Prerequisites: Go 1.19+
git clone https://github.com/aquasecurity/trivy.git
cd trivy
make build
sudo cp trivy /usr/local/bin/
```

Basic Usage

Container Image Scanning

```
# Scan Docker image
trivy image python:3.4-alpine

# Scan specific image with tag
trivy image nginx:1.21

# Scan local image
trivy image myapp:latest

# Scan image with specific format
trivy image --format json python:3.4-alpine

# Scan image and save results
trivy image --output results.json --format json python:3.4-alpine
```

File System Scanning

```
# Scan current directory
trivy fs .

# Scan specific directory
trivy fs /path/to/project

# Scan with specific file patterns
trivy fs --skip-files "*.test.js,*.spec.js" .

# Scan and include license information
trivy fs --license-full .
```

Repository Scanning

```
# Scan Git repository
trivy repo https://github.com/aquasecurity/trivy

# Scan specific branch
trivy repo --branch main https://github.com/aquasecurity/trivy

# Scan with authentication
trivy repo --username user --password token https://github.com/private/repo
```

Infrastructure as Code Scanning

```
# Scan Terraform files
trivy config terraform/

# Scan Kubernetes manifests
trivy config k8s-manifests/

# Scan CloudFormation templates
trivy config cloudformation/

# Scan Helm charts
trivy config helm-chart/

# Scan with specific policies
trivy config --policy custom-policies/ terraform/
```

Advanced Configuration

Severity Filtering

```
# Scan for HIGH and CRITICAL vulnerabilities only
trivy image --severity HIGH,CRITICAL python:3.4-alpine

# Exclude LOW severity
trivy image --severity MEDIUM,HIGH,CRITICAL nginx:latest

# Include all severities
trivy image --severity UNKNOWN,LOW,MEDIUM,HIGH,CRITICAL myapp:latest
```

Vulnerability Database Management

```
# Update vulnerability database
trivy image --download-db-only

# Skip database update
trivy image --skip-db-update python:3.4-alpine

# Use custom database
trivy image --cache-dir /custom/cache python:3.4-alpine

# Clear cache
trivy image --clear-cache
```

Output Formats

```
# JSON output
trivy image --format json python:3.4-alpine

# Table output (default)
trivy image --format table python:3.4-alpine

# SARIF output for GitHub
trivy image --format sarif python:3.4-alpine

# Template output
trivy image --format template --template "@contrib/gitlab.tpl" python:3.4-alpine

# CycloneDX SBOM
trivy image --format cyclonedx python:3.4-alpine

# SPDX SBOM
trivy image --format spdx-json python:3.4-alpine
```

Filtering and Ignoring

```
# Ignore specific vulnerabilities
trivy image --ignore-unfixed python:3.4-alpine

# Use .trivyignore file
echo "CVE-2021-12345" > .trivyignore
trivy image python:3.4-alpine

# Ignore specific file types
trivy fs --skip-files "*.md,*.txt" .

# Ignore specific directories
trivy fs --skip-dirs "node_modules,vendor" .
```

Configuration Files

.trivyignore File

```
# Create .trivyignore file
cat > .trivyignore << 'EOF'
# Ignore specific CVEs
CVE-2021-12345
CVE-2021-67890

# Ignore by package
pkg:npm/lodash@4.17.20

# Ignore by file path
/path/to/file.jar

# Ignore by severity
# (Note: Use command line for severity filtering)
EOF
```

Configuration File (trivy.yaml)

```yaml
# trivy.yaml
format: json
output: results.json
severity:
  - HIGH
  - CRITICAL
ignore-unfixed: true
skip-dirs:
  - node_modules
  - vendor
skip-files:
  - "*.test.js"
  - "*.spec.js"
cache:
  dir: /tmp/trivy-cache
db:
  skip-update: false
vulnerability:
  type:
    - os
    - library
```

Using Configuration File

```
# Use configuration file
trivy --config trivy.yaml image python:3.4-alpine

# Override configuration
trivy --config trivy.yaml --severity MEDIUM image python:3.4-alpine
```

CI/CD Integration

GitHub Actions

```yaml
# .github/workflows/trivy.yml
name: Trivy Security Scan

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  trivy-scan:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Run Trivy vulnerability scanner
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: 'myapp:latest'
        format: 'sarif'
        output: 'trivy-results.sarif'

    - name: Upload Trivy scan results to GitHub Security tab
      uses: github/codeql-action/upload-sarif@v2
      with:
        sarif_file: 'trivy-results.sarif'

    - name: Run Trivy for IaC
```

```
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: 'config'
        scan-ref: '.'
        format: 'sarif'
        output: 'trivy-iac-results.sarif'
```

GitLab CI

```yaml
# .gitlab-ci.yml
stages:
  - security

trivy-scan:
  stage: security
  image: aquasec/trivy:latest
  script:
    - trivy image --format template --template "@contrib/gitlab.tpl"
      --output gl-container-scanning-report.json myapp:latest
  artifacts:
    reports:
      container_scanning: gl-container-scanning-report.json
  only:
    - main
    - merge_requests

trivy-iac-scan:
  stage: security
  image: aquasec/trivy:latest
  script:
    - trivy config --format template --template "@contrib/gitlab-sast.tpl"
      --output gl-sast-report.json .
  artifacts:
    reports:
      sast: gl-sast-report.json
  only:
    - main
    - merge_requests
```

Jenkins Pipeline

```
// Jenkinsfile
pipeline {
    agent any

    stages {
        stage('Trivy Scan') {
            steps {
                script {
                    // Install Trivy
                    sh '''
                        curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b
/usr/local/bin
                    '''

                    // Scan Docker image
                    sh '''
                        trivy image --format json --output trivy-report.json myapp:latest
                    '''

                    // Scan IaC
                    sh '''
                        trivy config --format json --output trivy-iac-report.json .
                    '''
                }
            }
            post {
                always {
                    archiveArtifacts artifacts: 'trivy-*.json', fingerprint: true
                    publishHTML([
```

```
                        allowMissing: false,
                        alwaysLinkToLastBuild: true,
                        keepAll: true,
                        reportDir: '.',
                        reportFiles: 'trivy-report.json',
                        reportName: 'Trivy Security Report'
                    ])
                }
            }
        }
    }
}
```

Advanced Scanning Techniques

Multi-Stage Docker Scanning

```
# Scan specific stage in multi-stage build
trivy image --input myapp.tar

# Build and scan in pipeline
docker build -t myapp:latest .
trivy image myapp:latest

# Scan intermediate layers
docker save myapp:latest | trivy image --input -
```

Kubernetes Integration

```
# Scan Kubernetes cluster
trivy k8s cluster

# Scan specific namespace
trivy k8s --namespace kube-system cluster

# Scan specific resource
trivy k8s deployment/myapp

# Generate compliance report
trivy k8s --compliance k8s-cis cluster
```

SBOM Generation and Analysis

```
# Generate CycloneDX SBOM
trivy image --format cyclonedx --output sbom.json python:3.4-alpine

# Generate SPDX SBOM
trivy image --format spdx-json --output sbom-spdx.json python:3.4-alpine

# Scan SBOM file
trivy sbom sbom.json

# Convert between SBOM formats
trivy convert --format spdx-json --output converted.json sbom.json
```

Custom Policies and Rules

Creating Custom Policies

```
# Create custom policy directory
mkdir -p custom-policies

# Create Rego policy file
cat > custom-policies/custom-rule.rego << 'EOF'
package trivy.custom

import data.lib.result

deny[res] {
    input.kind == "Deployment"
    not input.spec.template.spec.securityContext.runAsNonRoot
    res := result.new("Container should not run as root", input.metadata)
}
EOF
```

Using Custom Policies

```
# Scan with custom policies
trivy config --policy custom-policies/ k8s-manifests/

# Combine with built-in policies
trivy config --policy custom-policies/ --policy builtin k8s-manifests/

# Validate policy syntax
trivy config --policy custom-policies/ --dry-run k8s-manifests/
```

Performance Optimization

Caching Strategies

```
# Set custom cache directory
export TRIVY_CACHE_DIR=/fast/storage/trivy-cache
trivy image python:3.4-alpine

# Parallel scanning
trivy image --parallel 4 python:3.4-alpine

# Skip unnecessary scans
trivy image --skip-files "*.md,*.txt" --skip-dirs "docs,examples" .
```

Database Optimization

```
# Download database separately
trivy image --download-db-only

# Use offline mode
trivy image --offline-scan python:3.4-alpine

# Custom database server
trivy image --db-repository custom-db-server.com/trivy-db python:3.4-alpine
```

Automation Scripts

Bulk Image Scanning

```bash
#!/bin/bash
# bulk-scan.sh

IMAGES_FILE="images.txt"
RESULTS_DIR="scan-results"

mkdir -p "$RESULTS_DIR"

while IFS= read -r image; do
    echo "Scanning $image..."
    output_file="$RESULTS_DIR/$(echo "$image" | tr '/:' '_').json"

    trivy image \
        --format json \
        --output "$output_file" \
        --severity HIGH,CRITICAL \
        "$image"

    if [ $? -eq 0 ]; then
        echo "✓ Scan completed for $image"
    else
        echo "✗ Scan failed for $image"
    fi
done < "$IMAGES_FILE"

echo "All scans completed. Results in $RESULTS_DIR/"
```

Automated Reporting

```python
#!/usr/bin/env python3
# trivy-reporter.py

import json
import sys
from datetime import datetime
import argparse

def parse_trivy_results(file_path):
    """Parse Trivy JSON results"""
    with open(file_path, 'r') as f:
        data = json.load(f)

    vulnerabilities = []
    if 'Results' in data:
        for result in data['Results']:
            if 'Vulnerabilities' in result:
                vulnerabilities.extend(result['Vulnerabilities'])

    return vulnerabilities

def generate_summary(vulnerabilities):
    """Generate vulnerability summary"""
    severity_counts = {'CRITICAL': 0, 'HIGH': 0, 'MEDIUM': 0, 'LOW': 0}

    for vuln in vulnerabilities:
        severity = vuln.get('Severity', 'UNKNOWN')
        if severity in severity_counts:
            severity_counts[severity] += 1
```

```
    return severity_counts

def generate_html_report(vulnerabilities, output_file):
    """Generate HTML report"""
    summary = generate_summary(vulnerabilities)

    html_content = f"""
    <!DOCTYPE html>
    <html>
    <head>
        <title>Trivy Security Report</title>
        <style>
            body {{ font-family: Arial, sans-serif; margin: 20px; }}
            .summary {{ background: #f5f5f5; padding: 15px; border-radius: 5px; }}
            .critical {{ color: #d32f2f; }}
            .high {{ color: #f57c00; }}
            .medium {{ color: #fbc02d; }}
            .low {{ color: #388e3c; }}
            table {{ border-collapse: collapse; width: 100%; margin-top: 20px; }}
            th, td {{ border: 1px solid #ddd; padding: 8px; text-align: left; }}
            th {{ background-color: #f2f2f2; }}
        </style>
    </head>
    <body>
        <h1>Trivy Security Report</h1>
        <p>Generated on: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}</p>

        <div class="summary">
            <h2>Summary</h2>
```

```
        <p><span class="critical">Critical: {summary['CRITICAL']}</span></p>
        <p><span class="high">High: {summary['HIGH']}</span></p>
        <p><span class="medium">Medium: {summary['MEDIUM']}</span></p>
        <p><span class="low">Low: {summary['LOW']}</span></p>
    </div>

    <h2>Detailed Vulnerabilities</h2>
    <table>
        <tr>
            <th>CVE ID</th>
            <th>Severity</th>
            <th>Package</th>
            <th>Installed Version</th>
            <th>Fixed Version</th>
            <th>Description</th>
        </tr>
"""

for vuln in vulnerabilities:
    html_content += f"""
        <tr>
            <td>{vuln.get('VulnerabilityID', 'N/A')}</td>
            <td class="{vuln.get('Severity', '').lower()}">{vuln.get('Severity', 'N/A')}</td>
            <td>{vuln.get('PkgName', 'N/A')}</td>
            <td>{vuln.get('InstalledVersion', 'N/A')}</td>
            <td>{vuln.get('FixedVersion', 'N/A')}</td>
            <td>{vuln.get('Description', 'N/A')[:100]}...</td>
        </tr>
    """
```

```
    html_content += """
        </table>
    </body>
    </html>
    """

    with open(output_file, 'w') as f:
        f.write(html_content)

def main():
    parser = argparse.ArgumentParser(description='Generate Trivy security reports')
    parser.add_argument('input_file', help='Trivy JSON results file')
    parser.add_argument('--output', '-o', default='trivy-report.html',
                        help='Output HTML file')

    args = parser.parse_args()

    try:
        vulnerabilities = parse_trivy_results(args.input_file)
        generate_html_report(vulnerabilities, args.output)
        print(f"Report generated: {args.output}")

        summary = generate_summary(vulnerabilities)
        print(f"Summary: {summary}")

    except Exception as e:
        print(f"Error: {e}")
        sys.exit(1)
```

```
if __name__ == '__main__':
    main()
```

CI/CD Integration Script

```bash
#!/bin/bash
# ci-trivy-scan.sh

set -e

# Configuration
IMAGE_NAME="${1:-myapp:latest}"
SEVERITY_THRESHOLD="${2:-HIGH,CRITICAL}"
OUTPUT_FORMAT="${3:-json}"
FAIL_ON_VULNERABILITIES="${4:-true}"

# Create results directory
mkdir -p trivy-results

# Scan container image
echo "Scanning container image: $IMAGE_NAME"
trivy image \
    --format "$OUTPUT_FORMAT" \
    --output "trivy-results/image-scan.json" \
    --severity "$SEVERITY_THRESHOLD" \
    "$IMAGE_NAME"

# Scan IaC files
echo "Scanning Infrastructure as Code files"
trivy config \
    --format "$OUTPUT_FORMAT" \
    --output "trivy-results/iac-scan.json" \
    --severity "$SEVERITY_THRESHOLD" \
    .
```

```
# Scan file system
echo "Scanning file system"
trivy fs \
    --format "$OUTPUT_FORMAT" \
    --output "trivy-results/fs-scan.json" \
    --severity "$SEVERITY_THRESHOLD" \
    .

# Generate summary report
python3 trivy-reporter.py trivy-results/image-scan.json --output trivy-results/image-report.html
python3 trivy-reporter.py trivy-results/iac-scan.json --output trivy-results/iac-report.html
python3 trivy-reporter.py trivy-results/fs-scan.json --output trivy-results/fs-report.html

# Check for vulnerabilities and fail if configured
if [ "$FAIL_ON_VULNERABILITIES" = "true" ]; then
    echo "Checking for vulnerabilities..."

    # Count vulnerabilities
    image_vulns=$(jq '[.Results[]?.Vulnerabilities[]? | select(.Severity == "HIGH" or .Severity == "CRITICAL")] | length'
trivy-results/image-scan.json)
    iac_vulns=$(jq '[.Results[]?.Misconfigurations[]? | select(.Severity == "HIGH" or .Severity == "CRITICAL")] | length'
trivy-results/iac-scan.json)
    fs_vulns=$(jq '[.Results[]?.Vulnerabilities[]? | select(.Severity == "HIGH" or .Severity == "CRITICAL")] | length' trivy-
results/fs-scan.json)

    total_vulns=$((image_vulns + iac_vulns + fs_vulns))

    echo "Found $total_vulns high/critical vulnerabilities"
```

```
    if [ "$total_vulns" -gt 0 ]; then
        echo "❌ Security scan failed: Found $total_vulns high/critical vulnerabilities"
        exit 1
    else
        echo "✅ Security scan passed: No high/critical vulnerabilities found"
    fi
fi

echo "Trivy scan completed successfully"
```

Monitoring and Alerting

Prometheus Metrics

```
# Export metrics for Prometheus
trivy image --format json python:3.4-alpine | \
jq -r '.Results[]?.Vulnerabilities[]? |
  "trivy_vulnerability{severity=\"" + .Severity + "\",cve=\"" + .VulnerabilityID + "\"} 1"'
```

Slack Notifications

```bash
#!/bin/bash
# slack-notify.sh

SLACK_WEBHOOK_URL="your-webhook-url"
SCAN_RESULTS="trivy-results.json"

# Parse results
critical_count=$(jq '[.Results[]?.Vulnerabilities[]? | select(.Severity == "CRITICAL")] | length' "$SCAN_RESULTS")
high_count=$(jq '[.Results[]?.Vulnerabilities[]? | select(.Severity == "HIGH")] | length' "$SCAN_RESULTS")

# Send notification
curl -X POST -H 'Content-type: application/json' \
    --data "{
        \"text\": \"Trivy Security Scan Results\",
        \"attachments\": [{
            \"color\": \"danger\",
            \"fields\": [{
                \"title\": \"Critical Vulnerabilities\",
                \"value\": \"$critical_count\",
                \"short\": true
            }, {
                \"title\": \"High Vulnerabilities\",
                \"value\": \"$high_count\",
                \"short\": true
            }]
        }]
    }" \
    "$SLACK_WEBHOOK_URL"
```

Troubleshooting

Common Issues

```
# Database update issues
trivy image --reset

# Clear cache
trivy image --clear-cache

# Debug mode
trivy image --debug python:3.4-alpine

# Check version
trivy version

# Verify installation
trivy image --help
```

Performance Issues

```
# Increase timeout
trivy image --timeout 10m python:3.4-alpine

# Reduce parallel processes
trivy image --parallel 1 python:3.4-alpine

# Skip database update
trivy image --skip-db-update python:3.4-alpine

# Use offline mode
trivy image --offline-scan python:3.4-alpine
```

Network Issues

```
# Use proxy
export HTTP_PROXY=http://proxy.example.com:8080
export HTTPS_PROXY=http://proxy.example.com:8080
trivy image python:3.4-alpine

# Custom registry
trivy image --registry-token token registry.example.com/myapp:latest

# Insecure registry
trivy image --insecure registry.example.com/myapp:latest
```

Best Practices

Security Scanning Strategy

1. **Shift Left**: Integrate Trivy early in development
2. **Multi-Layer Scanning**: Scan images, IaC, and file systems
3. **Continuous Monitoring**: Regular scans in CI/CD pipelines
4. **Severity Thresholds**: Define acceptable risk levels
5. **Remediation Workflow**: Establish vulnerability response process

Performance Optimization

1. **Cache Management**: Use persistent cache directories
2. **Parallel Processing**: Optimize for available resources
3. **Selective Scanning**: Skip unnecessary files and directories
4. **Database Management**: Keep vulnerability database updated
5. **Resource Allocation**: Allocate sufficient memory and CPU

Integration Guidelines

1. **CI/CD Integration**: Fail builds on high-severity vulnerabilities
2. **Reporting**: Generate actionable reports for teams
3. **Automation**: Automate scanning and reporting processes
4. **Monitoring**: Track vulnerability trends over time

5. **Compliance**: Align with security compliance requirements

This comprehensive Trivy cheatsheet provides everything needed for professional container and infrastructure security scanning, from basic usage to advanced automation and integration scenarios.